

✓ LABSHEET 7

```
from google.colab import files
```

```
uploaded = files.upload()
```

OnlineRetail.csv

OnlineRetail.csv(text/csv) - 45580638 bytes, last modified: 9/12/2026 - 100% done
Saving OnlineRetail.csv to OnlineRetail.csv

```
import pandas as pd
```

```
df = pd.read_csv("OnlineRetail.csv", encoding="latin1")
```

```
print(df.head())
```

```
print("\nDataset Shape:", df.shape)
```

```
print("\nColumn Names:")
```

```
print(df.columns)
```

	InvoiceNo	StockCode	Description	Quantity	\
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	
1	536365	71053	WHITE METAL LANTERN	6	
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	

	InvoiceDate	UnitPrice	CustomerID	Country
0	12/1/2010 8:26	2.55	17850.0	United Kingdom
1	12/1/2010 8:26	3.39	17850.0	United Kingdom
2	12/1/2010 8:26	2.75	17850.0	United Kingdom
3	12/1/2010 8:26	3.39	17850.0	United Kingdom
4	12/1/2010 8:26	3.39	17850.0	United Kingdom

```
Dataset Shape: (541909, 8)
```

```
Column Names:
```

```
Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',  
      'UnitPrice', 'CustomerID', 'Country'],  
      dtype='object')
```

Data Preprocessing

```
# Check missing values  
print("Missing Values:")  
print(df.isnull().sum())
```

```
# Remove duplicate rows  
df = df.drop_duplicates()
```

```
# Remove rows where CustomerID is missing
df = df.dropna(subset=["CustomerID"])

# Remove cancelled transactions
df = df[~df["InvoiceNo"].astype(str).str.startswith("C")]

# Remove invalid quantity and price values
df = df[(df["Quantity"] > 0) & (df["UnitPrice"] > 0)]

print("\nDataset after preprocessing:")
print(df.shape)
```

Missing Values:

InvoiceNo	0
StockCode	0
Description	1454
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	135080
Country	0

dtype: int64

Dataset after preprocessing:
(392692, 8)

Create Customer Features

```
# Calculate total amount spent
df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]

# Create customer-level features
customer_data = df.groupby("CustomerID").agg({
    "InvoiceNo": "nunique",
    "Quantity": "sum",
    "TotalAmount": "sum"
}).reset_index()

# Rename columns
customer_data.columns = [
    "CustomerID",
    "NumberOfOrders",
    "TotalQuantity",
    "TotalSpending"
]

print(customer_data.head())
print("\nCustomer Dataset Shape:", customer_data.shape)
```

	CustomerID	NumberOfOrders	TotalQuantity	TotalSpending
0	12346.0	1	74215	77183.60
1	12347.0	7	2458	4310.00
2	12348.0	4	2341	1797.24
3	12349.0	1	631	1757.55
4	12350.0	1	197	334.40

Customer Dataset Shape: (4338, 4)

Standardize Features

```
# Select features
features = customer_data[
    ["NumberOfOrders", "TotalQuantity", "TotalSpending"]
]

# Standardize the data
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)

print("Features standardized successfully!")
```

Features standardized successfully!

Elbow Method

```
inertia = []

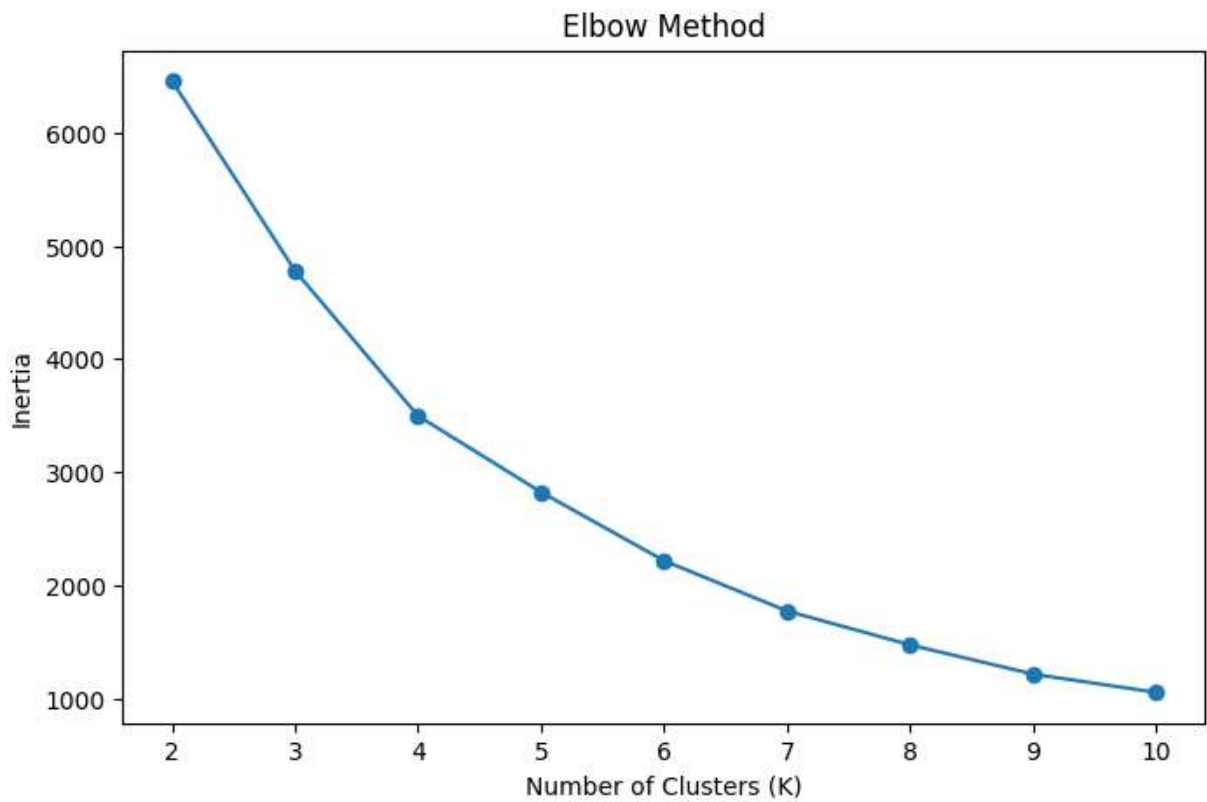
for k in range(2, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(scaled_features)
    inertia.append(kmeans.inertia_)

# Plot Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(range(2, 11), inertia, marker="o")

plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method")

plt.show()
```



Silhouette Score

```
silhouette_scores = []

for k in range(2, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = kmeans.fit_predict(scaled_features)
    score = silhouette_score(scaled_features, labels)

    silhouette_scores.append(score)

# Display scores
for k, score in zip(range(2, 11), silhouette_scores):
    print("K =", k, "Silhouette Score =", round(score, 4))
```

```
K = 2 Silhouette Score = 0.9613
K = 3 Silhouette Score = 0.9108
K = 4 Silhouette Score = 0.7683
K = 5 Silhouette Score = 0.7939
```

```

K = 6 Silhouette Score = 0.7686
K = 7 Silhouette Score = 0.6743
K = 8 Silhouette Score = 0.6538
K = 9 Silhouette Score = 0.651
K = 10 Silhouette Score = 0.6572

```

Apply K-Means

```

# Select number of clusters
k = 4

# Apply K-Means
kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

customer_data["Cluster"] = kmeans.fit_predict(scaled_features)

print(customer_data.head())

```

	CustomerID	NumberOfOrders	TotalQuantity	TotalSpending	Cluster
0	12346.0	1	74215	77183.60	3
1	12347.0	7	2458	4310.00	0
2	12348.0	4	2341	1797.24	0
3	12349.0	1	631	1757.55	0
4	12350.0	1	197	334.40	0

Visualize Clusters

```

plt.figure(figsize=(10, 6))

plt.scatter(
    customer_data["TotalSpending"],
    customer_data["TotalQuantity"],
    c=customer_data["Cluster"],
    s=50
)

# Convert cluster centers back to original scale
centers = scaler.inverse_transform(kmeans.cluster_centers_)

plt.scatter(
    centers[:, 2],
    centers[:, 1],
    marker="X",
    s=200,
    label="Centroids"
)

```

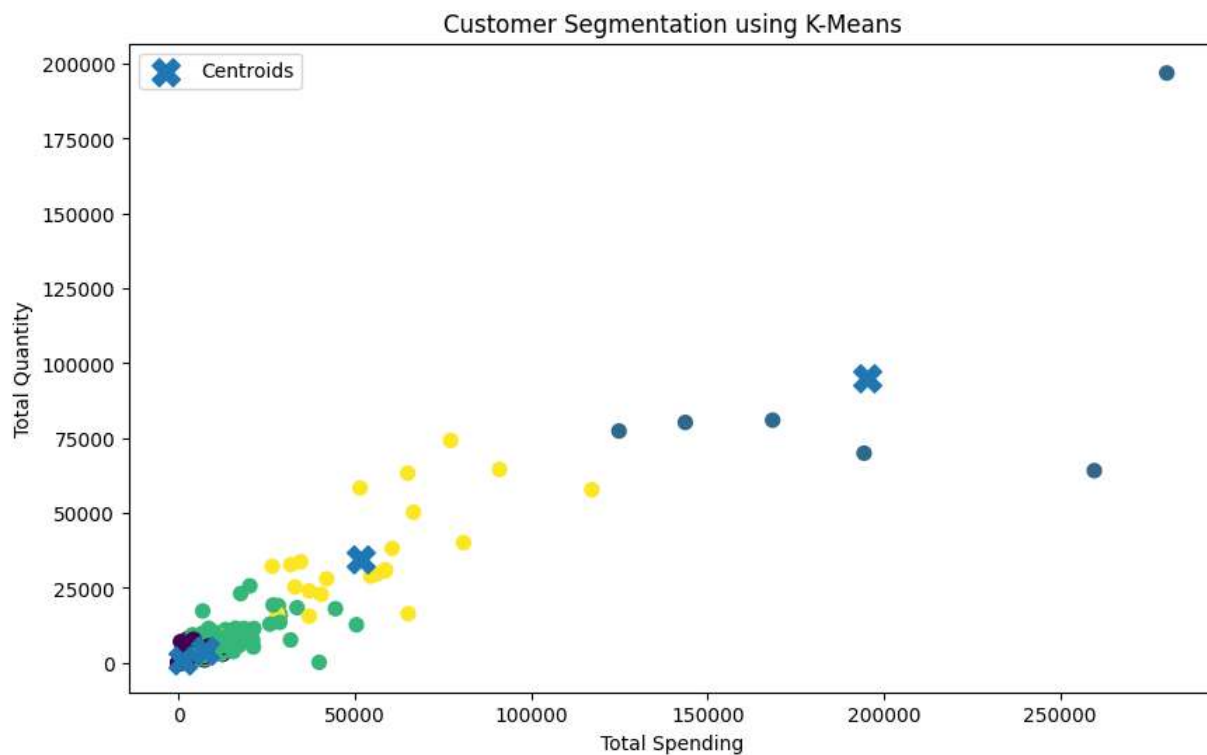
```

)

plt.xlabel("Total Spending")
plt.ylabel("Total Quantity")
plt.title("Customer Segmentation using K-Means")
plt.legend()

plt.show()

```



T B I <> 🔗 🖼️ “ ☰ ☷ — Ψ 😊 ☰ Close

Analyze Customer Segments

Analyze Customer Segments

```

cluster_summary = customer_data.groupby("Cluster").agg({
    "NumberOfOrders": "mean",
    "TotalQuantity": "mean",
    "TotalSpending": "mean"
}).round(2)

```

```
print("Customer Segment Analysis:")  
print(cluster_summary)
```

```
Customer Segment Analysis:  
      NumberOfOrders  TotalQuantity  TotalSpending  
Cluster  
0           2.82         587.95         983.23  
1          67.17       94937.17      195225.38  
2          16.42        4187.73        7597.37  
3          57.38       34548.54       51579.12
```

```
print("\nNumber of Customers in Each Cluster:")  
print(customer_data["Cluster"].value_counts().sort_index())
```

```
Number of Customers in Each Cluster:  
Cluster  
0      3969  
1         6  
2      339  
3        24  
Name: count, dtype: int64
```